Git Interview Questions and Answers: A Comprehensive Guide

This document contains a categorized collection of Git interview questions, ranging from foundational concepts to expert-level internals and DevOps best practices.

I. Basic-Level Questions

1. What is the difference between Git and SVN?

Answer: Git is a **distributed** version control system (DVCS) — every clone contains the full repository history, and most operations are performed locally. SVN (Subversion) is **centralized** — the history and commits are managed on a central server, and many operations require network access to communicate with that server.

2. What is the difference between `git pull` and `git fetch`?

Answer: `git fetch` downloads commits, files, and refs from a remote repository into your local repo but does **not** merge them into your working directory. `git pull` is a combination of `git fetch` followed immediately by `git merge` (or `git rebase`), which updates your current branch and working tree.

3. What is a "Merge Conflict" and how do you resolve it?

Answer: A merge conflict occurs when Git cannot automatically reconcile overlapping changes (e.g., the same line edited differently on two branches). To resolve it:

1. Open the affected files and locate the conflict markers (`<<<<<<<` , `=======` , `>>>>>>>`).
2. Edit the file to the desired final state.
3. Run `git add <file>` to mark it as resolved.
4. Run `git commit` to complete the merge.

4. What is the difference between `git merge` and `git rebase`?

Answer: - `git merge` : Combines histories and creates a new "merge commit." it preserves the true chronological history of how branches diverged and met.

- `git rebase` : Rewrites the commit history by moving your branch's commits to the tip of the target branch. This creates a cleaner, linear history but should be avoided on public/shared branches because it alters existing commit hashes.

5. What is `git stash` and when would you use it?

Answer: `git stash` temporarily "shelves" uncommitted changes (both staged and unstaged) to give you a clean working directory. This is useful when you need to switch branches quickly but aren't ready to commit your current work. Use `git stash pop` to bring the changes back.

6. What is the "Staging Area" (Index) in Git?

Answer: The staging area is an intermediate layer between the working directory and the repository. It allows you to format and review exactly what will go into your next commit. You use `git add` to move changes to the stage and `git commit` to save that snapshot.

7. What is `git cherry-pick` ?

Answer: `git cherry-pick <commit-hash>` applies the changes from a specific commit from one branch onto your current branch as a new commit. It is useful for backporting bug fixes without merging an entire branch.

8. How do you undo a commit that has already been pushed?

Answer: The safest way is to use `git revert <commit-hash>` , which creates a *new* commit that does the exact opposite of the target commit. This preserves history and is safe for shared branches. Using `git reset --hard` is dangerous on pushed branches as it deletes history.

9. What is a "Detached HEAD"?

Answer: A detached HEAD occurs when you check out a specific commit or tag instead of a branch. In this state, HEAD points directly to a commit hash rather than a branch pointer. Any new commits made here will not belong to a branch and may be lost unless you create a new branch immediately.

10. What is the purpose of `.gitignore` ?

Answer: `.gitignore` is a text file that tells Git which files or directories to ignore. This prevents unnecessary files like build artifacts (`/dist`), dependencies (`node_modules/`), or sensitive environment variables (`.env`) from being tracked.

11. What is `git clone` vs `git fork` ?

Answer: - `git clone` : A Git command that creates a local copy of an existing remote repository.

- `git fork` : A platform-level feature (GitHub/GitLab) that creates a personal copy of someone else's project on the server. You then clone your fork to work on it.

12. How do you find a bug-introducing commit?

Answer: Use `git bisect` . It performs a binary search through your commit history. You mark a "bad" commit (where the bug exists) and a "good" commit (where it didn't), and Git helps you test the midpoints until the offender is found.

13. What are Git Hooks?

Answer: Git hooks are scripts that run automatically every time a particular event occurs in a Git repository (e.g., `pre-commit` , `post-merge` , `pre-push`). They are stored in `.git/hooks/` and are used to automate linting, testing, or policy enforcement.

14. What is the difference between "Soft Reset" and "Hard Reset"?

Answer: - `--soft` : Moves HEAD to a different commit but leaves the index and working directory as they are. Your changes remain staged.

- `--hard` : Resets everything—HEAD, index, and working directory—to match the target commit. Any uncommitted changes are permanently lost.

15. What is a Pull Request (PR)?

Answer: A Pull Request is a feature of hosting services like GitHub that allows developers to notify team members that they have completed a feature. It provides a forum for code review, discussion, and automated testing before the code is merged into the main codebase.

II. Intermediate-Level Questions

16. What is the difference between `git reset`, `git revert`, and `git checkout`?

Answer: - `git reset` : Primarily used to move the branch pointer to a previous state (undoes local commits).

- `git revert` : Used to undo changes by creating a new inverse commit (safe for public history).
- `git checkout` : Used to switch between branches or restore files from the index/previous commits.

17. What is `git reflog` and why is it a lifesaver?

Answer: `git reflog` (Reference Log) tracks every single change to the HEAD pointer, even those not visible in `git log` (like deleted branches or hard resets). It allows you to find the SHA-1 of "lost" commits and recover them.

18. What is a "Squash" in Git?

Answer: Squashing is the process of combining multiple commits into a single one. This is usually done during an interactive rebase (`git rebase -i`) to clean up a "work-in-progress" history before merging a feature branch into the main branch.

19. Explain the "GitFlow" workflow.

Answer: GitFlow is a strict branching model:

- `main` : Production-ready code.
- `develop` : The main integration branch for features.
- `feature/` : Temporary branches for new features.
- `release/` : For preparing new production releases.
- `hotfix/` : For quick patches to production.

20. What is the difference between a "Fast-forward" merge and a "Three-way" merge?

Answer: - **Fast-forward**: Occurs if the target branch hasn't changed since you branched off. Git just moves the pointer forward.

- **Three-way**: Occurs if both branches have diverged. Git uses a common ancestor and the two branch tips to create a new merge commit.

21. What is `git commit --amend` ?

Answer: It allows you to modify the very last commit. You can change the commit message or add forgotten files to the previous commit. Like rebasing, it rewrites history, so don't use it on pushed commits.

22. What are "Dangling Commits"?

Answer: These are commits that are no longer reachable by any branch or tag. They often occur after a rebase or a reset. They stay in the database until Git's garbage collector (`git gc`) clears them out, or they can be recovered via `reflog` .

23. How do you find which developer changed a specific line in a file?

Answer: Use `git blame <filename>` . It displays the file line-by-line, showing the last commit ID, author, and timestamp for every single line.

24. What is `git submodule` ?

Answer: A tool that allows you to keep another Git repository as a subdirectory of your own. This is useful for including libraries or shared components while keeping their histories separate.

25. What happens if you delete a branch before merging it?

Answer: The branch pointer is removed. The commits themselves remain in the database as "dangling" commits for a while and can be recovered using the `reflog` if you have the SHA-1 hash, but they will eventually be deleted by garbage collection.

III. Expert-Level Questions

26. What are the four main types of objects in Git?

Answer: 1. **Blob**: Stores the file data (content). 2. **Tree**: Acts like a directory, mapping names to blobs or other trees. 3. **Commit**: Stores metadata (author, message, parent) and points to a root tree. 4. **Tag**: A named reference to a specific commit (annotated tags store additional metadata).

27. How does Git handle storage? Does it store "diffs" or "snapshots"?

Answer: Conceptually, Git stores **snapshots**. Every commit is a full picture of what the project looks like. However, for efficiency, Git uses **delta compression** in "packfiles." It stores the full content of one version and only the "diffs" for others to save disk space.

28. What is a "Git Mirror" (`--mirror`)?

Answer: A mirror clone is a "bare" clone that includes all remote-tracking branches and tags, and it maintains exactly the same layout as the source. It is typically used for backups or migrating a repository between hosting providers.

29. What is `git rev-parse` ?

Answer: It is a "plumbing" command used to resolve parameters. It can turn human-readable names (like `HEAD` or branch names) into their actual SHA-1 hashes. It is extensively used in automation scripts and Git hooks.

30. Explain "Atomic Commits" and why they matter in DevOps.

Answer: An atomic commit is a commit that contains exactly one logical change—including the code, the tests, and any necessary configuration.

- **Why it matters**:
  - **Reversibility**: If a bug is found, you can revert one commit without breaking unrelated features.
  - **Bisecting**: Makes `git bisect` much more effective at finding the exact cause of a failure.
  - **Code Review**: Smaller, focused commits are easier and faster to review.
  - **CI/CD**: Ensures that every step in the history is a "valid" state of the application that